

House Price Prediction

Executed project report with saved notebook outputs, tables, and charts

Project Description

House Price Prediction

Project Summary

This project is a clean, student-friendly implementation of a house price prediction. The main goal is to estimate house prices using property and location features. The notebook keeps the workflow simple: load the dataset, understand the columns, clean the data, train a baseline model, and check how well the model performs.

No external repository links or profile links are included in this description. Everything needed to understand and run the project is kept inside this folder.

Problem Statement

A raw dataset is useful only when it is converted into a clear decision-making workflow. In this project, the data is processed step by step so that important patterns can be found and a machine learning model can be trained. The expected output is based on price.

Files Included

File	Purpose	Quick Note
homedata.csv	Main data source for this folder	2000 sampled rows x 21 columns
Notebook file	Complete Python workflow	Includes loading, cleaning, training, and evaluation
PDF report	Human-readable project summary	Matches the updated project structure

Important Columns

Column	Role
id	Input feature used in the modelling workflow
date	Input feature used in the modelling workflow
price	Target / output column used in the modelling workflow
bedrooms	Input feature used in the modelling workflow
bathrooms	Input feature used in the modelling workflow
sqftliving	Input feature used in the modelling workflow
sqftlot	Input feature used in the modelling workflow
floors	Input feature used in the modelling workflow
waterfront	Input feature used in the modelling workflow
view	Input feature used in the modelling workflow
condition	Input feature used in the modelling workflow
grade	Input feature used in the modelling workflow

Workflow Followed

1. Load the data from the local dataset file present in the same project folder.
2. Inspect the structure using shape, column names, missing values, and basic statistics.
3. Clean the dataset by removing empty columns, handling missing values, and keeping only useful features.
4. Prepare the model input by separating features and the target column.
5. Train a baseline model using a simple scikit-learn pipeline with preprocessing.
6. Evaluate the result using practical metrics such as accuracy, classification report, MAE, RMSE, or R2 score depending on the target type.
7. Write observations in a short and direct way so the project can be explained easily.

How to Run

1. Open the notebook in Jupyter Notebook, JupyterLab, Google Colab, or VS Code.
2. Keep the dataset files in the same folder as the notebook.
3. Run the cells from top to bottom.
4. Read the printed metrics and notes at the end before making conclusions.

Final Note

This version keeps the logic understandable and avoids unnecessary complexity. The aim is not to make the notebook look overloaded, but to make it readable, explainable, and useful for a student-level data science portfolio.

Executed Notebook Outputs

The outputs below were generated from the saved notebook after running the code cells in this project folder.

Cell 2 - 2. Load the dataset

Dataset shape: (21613, 21)

```
id date price bedrooms bathrooms sqft_living \
0 7129300520 20141013T000000 227334 2 1.0097 1190
1 6414100192 20141209T000000 543328 2 2.2469 2580
2 5631500400 20150225T000000 183583 2 0.9975 779
3 2487200875 20141209T000000 598529 4 3.0154 1963
4 1954400510 20150218T000000 514642 3 2.0199 1666

sqft_lot floors waterfront view condition grade sqft_above \
0 5447 1.0069 0 0 4 6 1180
1 7295 1.9760 0 0 2 7 2179
2 10218 1.0076 0 1 4 6 758
3 5507 1.0149 0 1 6 8 1052
4 8181 1.0138 0 1 4 8 1684

sqft_basement yr_built yr_renovated zipcode lat long \
0 6 1956 3 98178 48.5262 -124.0406
1 403 1951 1988 98125 47.5240 -125.6399
2 3 1932 2 98027 46.7866 -123.4975
3 910 1964 0 98136 47.6450 -126.9533
4 0 1987 4 98074 47.2524 -119.5301

sqft_living15 sqft_lot15
0 1349 5734
1 1683 7306
2 2723 8162
3 1358 5403
4 1794 7324
```

Cell 3 - 3. Understand the data

Rows: 21613
Columns: 21

```
column dtype missing_values unique_values
0 id int64 0 21436
1 date object 0 372
2 price int64 0 21287
3 bedrooms int64 0 14
4 bathrooms float64 0 10039
5 sqft_living int64 0 3883
6 sqft_lot int64 0 12451
7 floors float64 0 4203
8 waterfront int64 0 2
9 view int64 0 6
10 condition int64 0 7
11 grade int64 0 13
12 sqft_above int64 0 3492
13 sqft_basement int64 0 1669
14 yr_built int64 0 118
15 yr_renovated int64 0 87
16 zipcode int64 0 129
17 lat float64 0 15850
18 long float64 0 18852
19 sqft_living15 int64 0 3125
20 sqft_lot15 int64 0 11670
```

Cell 4 - 4. Clean the data

Shape after simple cleaning: (21613, 21)

```
id date price bedrooms bathrooms sqft_living \
0 7129300520 20141013T000000 227334 2 1.0097 1190
1 6414100192 20141209T000000 543328 2 2.2469 2580
2 5631500400 20150225T000000 183583 2 0.9975 779
3 2487200875 20141209T000000 598529 4 3.0154 1963
4 1954400510 20150218T000000 514642 3 2.0199 1666

sqft_lot floors waterfront view condition grade sqft_above \
0 5447 1.0069 0 0 4 6 1180
1 7295 1.9760 0 0 2 7 2179
2 10218 1.0076 0 1 4 6 758
3 5507 1.0149 0 1 6 8 1052
4 8181 1.0138 0 1 4 8 1684

sqft_basement yr_built yr_renovated zipcode lat long \
0 6 1956 3 98178 48.5262 -124.0406
1 403 1951 1988 98125 47.5240 -125.6399
2 3 1932 2 98027 46.7866 -123.4975
3 910 1964 0 98136 47.6450 -126.9533
4 0 1987 4 98074 47.2524 -119.5301

sqft_living15 sqft_lot15
0 1349 5734
1 1683 7306
2 2723 8162
3 1358 5403
```

4 1794 7324

Cell 5 - 5. Select the target column

Selected target column: price

```
0 227334
1 543328
2 183583
3 598529
4 514642
Name: price, dtype: int64
```

Cell 6 - 6. Quick EDA

Numeric columns: 20
Categorical columns: 1
No missing values found in the previewed columns.

```
count mean std min \
id 21613.0 4.580302e+09 2.876566e+09 1.000102e+06
price 21613.0 5.400673e+05 3.671535e+05 7.647500e+04
bedrooms 21613.0 3.364318e+00 1.236576e+00 0.000000e+00
bathrooms 21613.0 2.114912e+00 7.713931e-01 0.000000e+00
sqft_living 21613.0 2.080012e+03 9.185723e+02 3.010000e+02
sqft_lot 21613.0 1.510403e+04 4.142074e+04 2.800000e+01
floors 21613.0 1.494301e+00 5.405878e-01 9.343000e-01
waterfront 21613.0 7.541757e-03 8.651720e-02 0.000000e+00
view 21613.0 5.381946e-01 8.459362e-01 0.000000e+00
condition 21613.0 3.418359e+00 1.045804e+00 0.000000e+00
grade 21613.0 7.656781e+00 1.437642e+00 0.000000e+00
sqft_above 21613.0 1.788374e+03 8.281073e+02 2.880000e+02
sqft_basement 21613.0 2.926450e+02 4.418540e+02 0.000000e+00
yr_built 21613.0 1.971016e+03 2.938283e+01 1.899000e+03
yr_renovated 21613.0 8.596706e+01 4.013641e+02 0.000000e+00

25% 50% 75% max
id 2.123049e+09 3.904930e+09 7.308900e+09 9.900000e+09
price 3.220840e+05 4.513660e+05 6.445830e+05 7.695399e+06
bedrooms 3.000000e+00 3.000000e+00 4.000000e+00 3.300000e+01
bathrooms 1.665900e+00 2.217600e+00 2.524300e+00 8.259700e+00
sqft_living 1.424000e+03 1.911000e+03 2.549000e+03 1.355100e+04
sqft_lot 5.096000e+03 7.646000e+03 1.072600e+04 1.651319e+06
floors 1.000300e+00 1.458800e+00 1.994500e+00 3.635700e+00
waterfront 0.000000e+00 0.000000e+00 0.000000e+00 1.000000e+00
view 0.000000e+00 0.000000e+00 1.000000e+00 5.000000e+00
condition 3.000000e+00 3.000000e+00 4.000000e+00 6.000000e+00
grade 7.000000e+00 8.000000e+00 9.000000e+00 1.400000e+01
sqft_above 1.192000e+03 1.562000e+03 2.211000e+03 9.400000e+03
sqft_basement 0.000000e+00 5.000000e+00 5.620000e+02 4.817000e+03
yr_built 1.951000e+03 1.975000e+03 1.997000e+03 2.016000e+03
yr_renovated 0.000000e+00 0.000000e+00 4.000000e+00 2.021000e+03
```

Cell 7 - 7. Prepare features and target

Dropped ID-like columns: ['id']
Using 1200 sampled rows for model training speed.
Problem type: regression
Feature shape: (1200, 19)
Target unique values: 1199

Cell 8 - 8. Train a baseline model

Training completed.

Cell 9 - 9. Evaluate the model

MAE : 170107.1445
RMSE: 254605.6517
R2 : 0.5939

